task3

任务完成

- 完成了 Music Transformer 训练 Notebook 的环境配置。
- 完成了依赖安装与设备检查（torch、torchaudio、encodec，CUDA 可用性）。

```
import torch
import torchaudio
import encodec

print("PyTorch version:", torch.__version__)
print("CUDA available:", torch.cuda.is_available())
```

[2] ✓ 0.8s

```
PyTorch version: 2.10.0
CUDA available: False
```

```
import os
os.environ["CUDA_LAUNCH_BLOCKING"] = "1"
```

[3] ✓ 0.0s

- 完成了自回归训练样本构建验证：将 [K, T] 转为 [T, K]，并通过 x=frames[:-1]、y=frames[1:] 对齐检查。

```
Original codes shape: torch.Size([4, 750])
Frames shape: torch.Size([750, 4])
x shape: torch.Size([749, 4])
y shape: torch.Size([749, 4])
x[:3]:
 tensor([[1019,  740,  710,  916],
        [ 228,  168,  894,  505],
        [ 999,  894,  752, 1018]])
y[:3]:
 tensor([[ 228,  168,  894,  505],
        [ 999,  894,  752, 1018],
        [ 228,  937,  752,   36]])
```

以上输出验证了 我们构建自回归训练样本 的正确性。

可以看到 x(输入) 的第2、3行 分别相同于 y(预测) 的第1、2行

即 输入x1 预测y1；输入x2 预测y2 。

- 完成了多 codebook 自回归 Transformer 的搭建。
- 4 本 codebook 独立 embedding。
- 位置编码 embedding。
- causal mask + TransformerEncoder。
- 输出分类头并还原到 [B, T, K, V]。
- 完成了模型前向 shape 测试，确认 logits 可用于每个时间步、每个 codebook 的交叉熵训练。

- 完成了 token 数据文件结构检查（.pt -> dict，包含 tokens 列表，每个 chunk 为 [4, 750]）。

```python
# 测试

model = MusicTransformer(
    vocab_size=1024,
    num_codebooks=x.shape[1],   # x的列数：4列（4本codebook）（749行，因为10s有750个音再-1）
    embed_dim=256,
    num_layers=6,
    num_heads=8
)

x_small = x[:512]   # 只取512长度
dummy_x = x_small.unsqueeze(0)
logits = model(dummy_x)

print(f"Number of codebooks: {len(logits)}") # logits打分矩阵的行数：4行，分别对应4本codebook里的所有音
print("Logits shape for codebook 0:", logits[0].shape)
```

[9] ✓ 1.2s Python

```
Number of codebooks: 1
Logits shape for codebook 0: torch.Size([512, 4, 1024])
```

```python
import os
import torch
import random

TOKEN_DIR = "maestro_tokens_3kbps"

song_files = sorted([
    os.path.join(TOKEN_DIR, f)
    for f in os.listdir(TOKEN_DIR)
    if f.endswith(".pt")
])

print(f"Found {len(song_files)} songs.")
```

[10] ✓ 0.0s Python

```
Found 17 songs.
```

```python
# 看看我们存的.pt文件是什么 是dict还是tensor

data = torch.load("maestro_tokens_3kbps/music_0000.pt")
print(type(data))
print(data.keys())
```

[11] ✓ 0.0s Python

```
<class 'dict'>
dict_keys(['tokens', 'num_chunks', 'bandwidth', 'sample_rate', 'chunk_size', 'music_id'])
```

```python
data = torch.load("maestro_tokens_3kbps/music_0000.pt")

print(type(data["tokens"]))
print(len(data["tokens"]))
print(type(data["tokens"][0]))
print(data["tokens"][0].shape)
```

[12] ✓ 0.0s Python

```
<class 'list'>
30
<class 'torch.Tensor'>
torch.Size([4, 750])
```

- 完成了 Dataset 构建：整首拼接、滑窗切片、生成 (x, y) 样本。
- 使用 stride=256 的重叠切片，提高样本覆盖。

- 完成了完整训练流程配置：DataLoader、AdamW、warmup + linear decay、梯度裁剪。
- 完成了多 codebook 加权损失训练策略。
- weights = [1.0, 0.5, 0.25, 0.1]。
- 强化低频主结构（CB1）学习。
- 完成了 checkpoint 保存流程，可按 epoch 持久化模型与优化器状态。
- 完成了 Music Transformer 生成 Notebook 的环境配置。
- 完成了依赖安装与设备检查（torch、torchaudio、encodec、torchcodec、transformers）。
- 完成了生成模型加载流程。
- 从 `music_transformer.py` 导入 `MusicTransformer`。
- 按训练阶段一致的超参数构建模型。
- 加载 `maestro_checkpoints/checkpoint_epoch4.pt` 权重并切换 `eval()` 推理模式。
- 完成了自回归生成函数搭建（`generate`）。
- 每一步仅使用可见历史 token（裁剪到 `max_seq_len`）。
- 对每个 codebook 的最后一步 logits 做 `temperature` 缩放。
- 使用 `softmax + multinomial` 进行采样并拼接新 token。
- 完成了 prompt 驱动生成实验。
- 从 `MusicTokenDataset` 取样本。
- 以前 10 帧作为起始 prompt。
- 生成 400 个新时间步，`temperature=1.1`。
- 完成了生成结果的离散 token 质量观察。
- 统计 4 本 codebook 各自使用的不同 token 数量，作为多样性参考指标。
- 完成了 Encodec 解码流程。
- 使用 `EncodecModel.encodec_model_24khz()`。
- 设置 `set_target_bandwidth(3.0)`（3kbps）。
- 将生成 token 还原为波形并得到 `wav`。
- 完成了生成音频试听与导出。
- 使用 `IPython.display.Audio` 在线试听。
- 使用 `soundfile` 保存为 `generated_music.wav`。

学习心得

这次训练 Notebook 把“音频 token 序列建模”完整串了起来：从样本构建、模型设计到训练与保存，形成了可复现的闭环流程。我对多 codebook 建模的理解更清晰了：不同 codebook 的语义空间不同，独立 embedding 更合理；同时使用因果 mask 能保证严格自回归训练，不泄露未来信息。另外，训练工程细节（AdamW、warmup、梯度裁剪、分 codebook 加权）对稳定收敛非常关键。后续我会继续补充验证集评估与生成质量对比，让训练结果从“可训练”走向“可评估、可迭代优化”。
这次 Notebook 把“训练好的离散 token 模型如何真正发声”跑通了完整链路：`checkpoint ->`

`token 采样 -> codec 解码 -> 音频试听/保存`，从而把训练阶段和应用阶段连接起来了。我对生成阶段的核心控制点有了更清晰的认识：`temperature` 直接影响采样分布的尖锐程度，数值偏低会更稳但更保守，数值偏高会更活跃但也更容易出现不稳定片段。另外，按 codebook 观察 token 多样性是一个很实用的快速诊断方法。后续我会继续尝试 `top-k / top-p` 采样和更长 prompt，比较不同采样策略下音乐连贯性与新颖性的平衡。